ML4 - Compression of ML Models

Luka Capan[1], Borna Budić[1], Silvia Petrović[1], Jakov Dragičević[1]

***Abstract*— Deep neural networks (DNNs) achieve remarkable results on image classification tasks, yet their growing complexity makes deployment on resource-constrained devices challenging. This paper presents a practical implementation of a complete compression pipeline applied to a convolutional neural network (CNN) trained from scratch on the Fruits-360 dataset at 100x100 pixel resolution. The pipeline combines three complementary techniques: an improved architecture with BatchNorm and Dropout layers, dynamic weight pruning during training, and post-training INT8 quantization. The entire implementation is realized exclusively in the NumPy library, without the use of any deep learning framework. The results show that the improved architecture achieves 77.55% top-1 accuracy across 261 classes, an improvement of 5.25 percentage points over the original baseline (72.30%) while using 52% fewer parameters. Applying dynamic pruning preserves accuracy at 77.45% at 50% sparsity, and a subsequent INT8 quantization further reduces the theoretical model size 4× with an accuracy drop of only 0.05%. The final compressed model is 16.7× smaller than the original baseline while simultaneously achieving higher accuracy.**

I. INTRODUCTION

The rapid advancement of deep learning has produced ever more powerful neural network models, but this progress comes at a cost: modern DNNs require significant memory and computational resources that are often unavailable on edge devices such as smartphones, microcontrollers, and FPGAs [1]. For example, ResNet-50 requires 98 MB of storage and over 3.8 billion FLOPs to process a single image.

Model compression addresses this problem by reducing the size and computational complexity of a trained network while preserving as much predictive accuracy as possible. The reference paper by Li, Li, and Meng [1] identifies five main families of compression techniques: weight pruning, parameter quantization, low-rank decomposition, knowledge distillation, and lightweight model design.

The goal of this paper is to implement and evaluate a complete compression pipeline that combines three complementary techniques: *(i)* architectural design with embedded regularization (see Section 4 of [1]), *(ii)* dynamic weight pruning (see Section 2.2 of [1]), and *(iii)* post-training quantization (see Section 3 of [1]). The implementation uses only NumPy, which makes every mathematical operation explicit and directly tied to the equations from the reference paper.

The main contributions of this paper are:

- A complete CNN training pipeline in pure NumPy, including forward propagation, backpropagation, and SGD with momentum.
- An improved architecture with BatchNorm and Dropout layers that reduces the parameter count by 52%.
- Dynamic global unstructured weight pruning by L1 magnitude that gradually increases sparsity during training.
- Post-training INT8 symmetric and asymmetric quantization following Algorithm 1 and Equations (1)–(5) from [1].
- A quantitative comparison of accuracy, model size, and inference time across the entire compression pipeline.

[1]Faculty of Engineering, University of Rijeka, Croatia

II. METHODOLOGY

A. Network Architecture — Baseline

The initial architecture, called *SimpleCNN*, consists of two convolutional blocks followed by two fully connected layers:

1) **Conv1**: 8 filters, 3×3 kernel, padding $1 \rightarrow$ ReLU $\rightarrow$ MaxPool 2×2
2) **Conv2**: 16 filters, 3×3 kernel, padding $1 \rightarrow$ ReLU $\rightarrow$ MaxPool 2×2
3) **Flatten**: $(N,\ 16,\ 25,\ 25) \rightarrow (N,\ 10000)$
4) **FC1**: $10000 \rightarrow 128$, ReLU
5) **FC2**: $128 \rightarrow C$ (number of classes), Softmax

Input images are RGB at 100x100 resolution, normalized to $[0, 1]$. The total number of parameters is approximately 1.32 million, of which 1.28 million belongs to the FC1 layer. He initialization is applied to all convolutional and fully connected layers.

B. Improved Architecture

Analysis of the baseline model revealed significant overfitting: training accuracy reaches approximately 99% while test accuracy plateaus around 72%. A gap of roughly 27 percentage points indicates that the model relies too heavily on memorizing the training set. To address this problem, an improved architecture was designed with three concrete changes:

1) **BatchNorm layer after each convolution** – normalizes activations within a batch, stabilizes training, and acts as a mild regularizer [4].
2) **Dropout with rate $p = 0.3$ after FC1** – during training, randomly sets 30% of activations to zero, preventing FC1 from memorizing individual examples.
3) **A third convolutional block (16→32 filters)** – adds depth and, as a consequence, a third MaxPool layer that reduces the spatial dimension from 25×25 to 12×12. This reduces the FC1 input from 10 000 to 4 608 features, and FC1 itself from 1.28M to 590K parameters.

The improved network has a total of 629 765 parameters, a 52% reduction relative to the baseline. The public interface of the improved model is identical to the baseline interface (forward, backward, get_trainable_layers), which ensures that training, pruning, and quantization work without changes to the rest of the code.

C. Dynamic Weight Pruning

Global unstructured L1 pruning has been implemented following Han et al., as described in Section 2.2 [1]. Unlike the classical *train* $\rightarrow$ *prune* $\rightarrow$ *fine-tune* paradigm, the **dynamic pruning** applied here gradually increases sparsity *during* training according to the schedule:

$$s(t) = s_{\text{final}} \cdot \left(1 - \left(1 - \frac{t - t_0}{t_1 - t_0}\right)^3\right) \tag{1}$$

where $s(t)$ is the current sparsity level at epoch t, $s_{\text{final}} = 0.5$, $t_0 = 2$ (start epoch), and $t_1 = 15$ (end epoch). The cubic curve provides faster growth of sparsity in the early epochs and a slowdown near the end.

After each gradient update, the binary mask is re-applied to ensure pruned weights remain exactly zero. The gradual approach allows the model to adapt to the loss of weights instead of suddenly losing 50% of capacity.

D. Post-Training Quantization

Symmetric and asymmetric INT8 quantization is implemented following Algorithm 1 and Equations (1)–(5) [1]. For each layer separately, the scale factor S and zero point Z are computed from the weight range $[R_{\min}, R_{\max}]$:

$$S = \frac{R_{\max} - R_{\min}}{Q_{\max} - Q_{\min}}, \quad Z = Q_{\max} - \frac{R_{\max}}{S} \tag{2}$$

Weights are then quantized to INT8:

$$Q = \text{round}\left(\frac{R}{S} + Z\right), \quad Q \in [-128,\ 127] \tag{3}$$

and dequantized back to FP32 for inference:

$$\hat{R} = (Q - Z) \cdot S \tag{4}$$

The round-trip FP32$\rightarrow$INT8$\rightarrow$FP32 procedure enables measuring the impact of quantization on accuracy while the forward pass still uses FP32 arithmetic. In a real hardware deployment storing weights as INT8 (1 byte instead of 4), this yields a theoretical $4\times$ reduction in model size.

III. EXPERIMENTAL SETUP

A. Dataset

We used *Fruits-360* [2], a publicly available dataset of fruit images photographed against a white background. Each image has a resolution of 100x100 pixels in RGB format. The version used contains 261 different classes of fruits and vegetables.

For the experiments, 100 images per class were used for training and the full test split was used for evaluation (approximately 26 016 training and 25 244 test images). Images were normalized to $[0, 1]$ by dividing pixel values by 255. No data augmentation was applied.

B. Experimental Procedure

The complete compression pipeline is executed in three steps:

1) **Training the improved architecture with dynamic pruning**: the improved CNN is trained for 15 epochs with $\eta = 0.01$, batch size 32, momentum 0.9. Pruning starts in epoch 2 and gradually reaches 50% sparsity by epoch 15.
2) **Quantization**: the best checkpoint from step 1 is loaded, and asymmetric INT8 PTQ is then applied layer by layer.
3) **Evaluation**: accuracy is measured on the full test set after each step.

The pipeline was run on a laptop with 16 GB of RAM without GPU acceleration. The total duration of the full run is approximately 5 hours and 30 minutes.

IV. RESULTS AND DISCUSSION

A. Comparison of the Baseline and Improved Architectures

Table I shows the results of the comparison between the baseline and the improved architectures under identical training conditions (same seed, same dataset, 10 epochs, 100 images per class).

TABLE I
COMPARISON OF THE BASELINE AND IMPROVED ARCHITECTURES UNDER IDENTICAL CONDITIONS.

Metric	Baseline	Improved
Top-1 accuracy (%)	72.30	**77.55**
Number of parameters	1 315 189	**629 765**
Model size (KB)	10 275	4 920
Inference time (ms)	82.30	89.14
Training time	2h 34min	3h 16min
Accuracy change	—	**+5.25%**
Parameter count change	—	**−52.1%**

The improved architecture achieves a 5.25 percentage point improvement in accuracy while simultaneously reducing the parameter count by 52.1%. Although inference is 8.3% slower due to BatchNorm operations, and training is 27% longer due to the additional convolutional layer, both of these costs are one-time and acceptable for the improvement in generalization.

B. Dynamic Pruning on the Improved Model

Table II shows the progression of training with dynamic pruning across 15 epochs.

The best model was achieved at epoch 14 with 77.45% accuracy at a sparsity of 49.98%. It is important to note that pruning practically did not harm accuracy: the improved model without pruning reaches 77.55%, and with 50% pruning reaches 77.45% — a difference of only 0.10 percentage points. The gradual nature of dynamic pruning allowed the

TABLE II

PROGRESSION OF DYNAMIC PRUNING DURING TRAINING OF THE IMPROVED MODEL.

Epoch	Loss	Accuracy (%)	Sparsity (%)
1	5.04	9.39	0.00
3	1.95	64.25	18.44
5	0.94	71.73	31.75
7	0.60	74.31	40.67
10	0.36	75.89	47.72
12	0.27	76.81	49.50
14	0.23	**77.45**	49.98
15	0.20	77.26	50.00

model to adapt to the loss of weights instead of having to tolerate a sudden reduction in capacity.

C. Final Compression with Quantization

After quantizing the pruned improved model to INT8, the results shown in Table III were obtained.

TABLE III

RESULTS OF INT8 QUANTIZATION APPLIED TO THE PRUNED IMPROVED MODEL.

Metric	Pruned (FP32)	Quantized (INT8)
Accuracy (%)	76.53	76.48
Size (KB)	4 920	**614.5** (theor.)
Inference time (ms)	87.36	**73.60**
Accuracy drop	—	0.05%
Speedup	—	1.19×

INT8 quantization achieves a theoretical 4× reduction in model size with a negligible accuracy drop of 0.05%. The measured inference speedup of 1.19× is a consequence of the reduced dynamic range of weights after quantization.

D. Complete Comparison: Baseline → Improved → Pruning → Quantization

Table IV shows the final comparison of all stages across the entire compression pipeline.

TABLE IV

COMPLETE COMPARISON OF ALL COMPRESSION STAGES RELATIVE TO THE BASELINE.

Stage	Accuracy (%)	Size (KB)	Factor
Baseline	72.30	10 275	1.0×
Improved (BN + Dropout)	77.55	4 920	2.1×
Improved + Pruning	77.45	2 460*	4.2×
Improved + Pruning + INT8	**76.48**	**614.5**	**16.7**×

*Effective size given 50% zeros in the weights.

The final compressed model is **16.7× smaller** than the original baseline, while simultaneously achieving **4.18 percentage points higher accuracy** (76.48% vs. 72.30%). This result shows that a well-designed combination of architectural design and compression techniques can simultaneously improve and condense a model, consistent with the recommendations of the reference paper [1].

E. Per-layer Analysis of Quantization Error

The average quantization error per layer (mean absolute difference between original and dequantized weights) remained below 0.003 for all layers. The smallest error was observed in the FC1 layer (0.000377), confirming that the 256-level INT8 grid is more than fine enough for the weight distributions observed in the fully connected layers.

V. CONCLUSION

This paper presented a complete compression pipeline for CNN models implemented in pure NumPy, combining three complementary techniques described in the reference paper by Li, Li, and Meng [1].

The main results are as follows. The improved architecture with BatchNorm and Dropout layers achieved 77.55% top-1 accuracy with 52% fewer parameters than the baseline. Dynamic L1 pruning at 50% sparsity preserved accuracy at 77.45% (a drop of only 0.10 percentage points). Post-training INT8 quantization achieved a theoretical 4× reduction in size with an accuracy drop of 0.05% and an inference speedup of 1.19×.

The combination of all three techniques yields a final model that is 16.7× smaller than the original baseline while simultaneously achieving 4.18 percentage points *higher* top-1 accuracy. This demonstrates that careful architectural design combined with dynamic pruning and INT8 quantization can achieve significant compression without any significant loss of predictive power.

Directions for future work include experimentation with lower quantization levels (INT4), structured pruning for hardware-friendly acceleration, and applying knowledge distillation as a fourth complementary technique.

REFERENCES

[1] Z. Li, H. Li, and L. Meng, "Model Compression for Deep Neural Networks: A Survey," *Computers*, vol. 12, no. 3, p. 60, March 2023. https://doi.org/10.3390/computers12030060

[2] H. Mureşan and M. Oltean, "Fruit recognition from images using deep learning," *Acta Universitatis Sapientiae, Informatica*, vol. 10, no. 1, pp. 26–42, 2018.

[3] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. NeurIPS*, Montreal, QC, Canada, 2015, pp. 1135–1143.

[4] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, Lille, France, 2015, pp. 448–456.

[5] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.